

Reinforcement Learning: From Foundations to CSAC

A Step-by-Step Mathematical Guide

August 2026

Roadmap

- 1 Reinforcement Learning Setup
- 2 Value Functions & Bellman Equations
- 3 Value Iteration & Deep Q-Learning
- 4 Policy Optimization & Policy Gradients
- 5 Actor-Critic Methods
- 6 Entropy & Soft Actor-Critic (SAC)
- 7 Constrained Soft Actor-Critic (CSAC)

The MDP Framework

Markov Decision Process (MDP) Definition

A reinforcement learning problem is formulated as a 5-tuple MDP: $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$

- $\mathcal{S}$: State space $\{s_0, s_1, \dots\}$ representing all possible environment configurations.
- $\mathcal{A}$: Action space $\{a_0, a_1, \dots\}$ denoting available agent decisions.
- $P(s_{t+1} \mid s_t, a_t)$: Transition probability density function dictating state progression.
- $R(s_t, a_t, s_{t+1}) = r_t$: Scalar feedback reward indicating performance quality.
- $\gamma \in [0, 1)$: Discount factor ensuring convergence and balancing immediate vs. future rewards.

Agent-Environment Dynamics & Markov Property

Sequential Interaction Cycle

At time step t , the agent observes state $s_t \in \mathcal{S}$, selects action $a_t \sim \pi(\cdot \mid s_t)$, receives reward r_t , and transitions to state $s_{t+1} \sim P(\cdot \mid s_t, a_t)$.

The Markov Property

Future dynamics depend **only** on the current state and action, independent of history:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t)$$

- State s_t must summarize all historical information relevant to future transitions.
- The entire trajectory is defined as $\tau = (s_0, a_0, s_1, a_1, \dots)$.

Policy, Return, and Objective

Stochastic Policy & Discounted Return

A policy defines an action distribution conditioned on states: $\pi(a \mid s) = P(a_t = a \mid s_t = s)$.
The discounted cumulative return starting from step t is:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} = r_t + \gamma G_{t+1}$$

Reinforcement Learning Objective

Find the optimal policy π^* that maximizes the expected discounted return over trajectories:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \implies \pi^* = \arg \max_{\pi} J(\pi)$$

State and Action-Value Functions

State-Value Function $V^\pi(s)$

Measures the expected discounted return starting from state s under policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid s_t = s]$$

Action-Value Function $Q^\pi(s, a)$

Measures the expected return when starting from s , taking arbitrary action a , then following π :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid s_t = s, a_t = a]$$

- Connection between V and Q : $V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot | s)} [Q^\pi(s, a)] = \sum_{a \in \mathcal{A}} \pi(a | s) Q^\pi(s, a)$
- **Advantage Function:** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ indicates how much better action a is over the policy's average action.

Bellman Expectation Equations

By decomposing $G_t = r_t + \gamma G_{t+1}$, we derive recursive Bellman expectation equations:

State-Value Expectation

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s), s' \sim P(\cdot|s,a)} [R(s, a, s') + \gamma V^\pi(s')]$$

Action-Value Expectation

$$Q^\pi(s, a) = \mathbb{E}_{s' \sim P(\cdot|s,a)} [R(s, a, s') + \gamma V^\pi(s')]$$

- Forms the computational foundation for policy evaluation via dynamic programming and Temporal Difference (TD) learning.

Bellman Optimality Equations

Instead of evaluating a fixed π , define optimal values: $V^*(s) = \max_{\pi} V^{\pi}(s)$ and $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$.

Bellman Optimality Equations

$$V^*(s) = \max_{a \in \mathcal{A}} \mathbb{E}_{s' \sim P(\cdot|s,a)} [R(s, a, s') + \gamma V^*(s')]$$

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

Policy Extraction

Given $Q^*(s, a)$, an optimal deterministic policy is recovered greedily:

$$\pi^*(s) \in \arg \max_{a \in \mathcal{A}} Q^*(s, a)$$

Dynamic Programming to Function Approximation

Tabular Value Iteration

Applies the contraction mapping repeatedly from an arbitrary initial $V_0(s)$:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \mathbb{E}_{s' \sim P(\cdot | s, a)} [R(s, a, s') + \gamma V_k(s')]$$

Guaranteed to converge as $k \rightarrow \infty$: $V_k(s) \rightarrow V^*(s)$.

The Continuous State Space Problem

- Tabular storage fails when $|\mathcal{S}|$ is infinite/continuous.
- **Solution:** Approximate optimal value functions via parameterized models (e.g., neural networks)
 $V_\phi(s) \approx V^*(s)$ or $Q_\phi(s, a) \approx Q^*(s, a)$.

Deep Fitted Value / Q-Iteration

Deep Q-Learning Objective

Approximate $Q^*(s, a)$ with network parameters ϕ . For transition (s_t, a_t, r_t, s_{t+1}) , define the 1-step target:

$$y_t = r_t + \gamma \max_{a'} Q_{\phi}(s_{t+1}, a')$$

Update parameters ϕ by minimizing the Mean Squared Bellman Error:

$$L(\phi) = \mathbb{E} \left[(Q_{\phi}(s_t, a_t) - y_t)^2 \right]$$

- Target depends on current parameters ϕ , causing non-stationary regression targets (mitigated by target networks).
- Policy is derived greedily: $\pi^*(s) = \arg \max_a Q_{\phi}(s, a)$.

Direct Policy Parameterization

Motivation

Value-based methods struggle in continuous action spaces (computing $\max_a Q(s, a)$ is intractable) and cannot represent stochastic policies.

Policy Optimization Setup

Parameterize the policy directly: $\pi_\theta(a | s)$ (e.g., a Gaussian policy with neural network parameters θ).

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Optimize via gradient ascent: $\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta_k)$.

The Policy Gradient Theorem

Analytical Gradient

Because environment transitions $P(s' | s, a)$ are unknown, we leverage the Policy Gradient Theorem to differentiate through expectations:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi_{\theta}}(s_t, a_t) \right]$$

The Actor-Critic Paradigm

Actor-Critic algorithms combine the sample efficiency of value functions with the continuous action handling of policy gradients:

The Actor (π_θ)

- Controls behavior: $a_t \sim \pi_\theta(\cdot \mid s_t)$.
- Updated in the direction of higher advantage via policy gradient.

The Critic (V_ϕ or Q_ϕ)

- Evaluates current policy: $V_\phi(s) \approx V^{\pi_\theta}(s)$.
- Uses bootstrapping to reduce variance.

Temporal Difference (TD) Advantage

The 1-step TD error serves as an unbiased estimate of the advantage function:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \approx \hat{A}_t$$

Actor-Critic Architecture & Training Loop

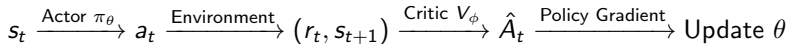
Alternating Optimization

- ① **Critic Step:** Minimize TD error loss:

$$L_V(\phi) = \mathbb{E} \left[(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))^2 \right] \implies \phi \leftarrow \phi - \alpha_\phi \nabla_\phi L_V(\phi)$$

- ② **Actor Step:** Update along estimated advantage:

$$\theta \leftarrow \theta + \alpha_\theta \mathbb{E} \left[\nabla_\theta \log \pi_\theta(a_t | s_t) \hat{A}_t \right]$$



Limitations of Standard AC & Maximum Entropy

Why Standard Actor-Critic Fails

Standard policy gradient methods quickly become near-deterministic when early actions receive high rewards. This causes:

- Premature convergence to suboptimal local equilibria.
- Poor exploration in complex, multimodal reward landscapes.

Maximum Entropy RL Framework

Augment the standard objective with an information-theoretic entropy term $\mathcal{H}(\pi)$:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right]$$

where policy entropy is $\mathcal{H}(\pi(\cdot | s)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s)]$, and $\alpha > 0$ balances reward vs. exploration.

Soft Value Functions and Targets

Soft State and Action-Value Functions

Under the maximum entropy framework, value functions incorporate entropy:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [Q^\pi(s, a) - \alpha \log \pi(a \mid s)]$$

$$Q^\pi(s, a) = \mathbb{E}_{s' \sim P} [r(s, a) + \gamma V^\pi(s')]$$

Soft Bellman Target with Target Networks

Using parameterized target critics $Q_{\bar{\phi}}$:

$$y_t = r_t + \gamma \mathbb{E}_{a_{t+1} \sim \pi_\theta} [Q_{\bar{\phi}}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\theta(a_{t+1} \mid s_{t+1})]$$

Soft Policy Improvement via KL Divergence

Optimal Soft Policy

For a given soft Q -function, the maximum-entropy optimal policy has the Boltzmann form:

$$\pi^*(a | s) \propto \exp \left(\frac{Q(s, a)}{\alpha} \right)$$

Parameterized Policy

We represent the policy with a neural network:

$$\pi_{\theta}(a | s)$$

and fit it to the ideal soft policy by minimizing the KL divergence:

$$\pi_{\theta}^* = \arg \min_{\pi_{\theta}} D_{\text{KL}} (\pi_{\theta}(\cdot | s) \| \pi^*(\cdot | s))$$

Soft Actor-Critic: Practical Algorithm

Clipped Double-Q Trick

To mitigate positive overestimation bias in the critic, SAC maintains two critics Q_{ϕ_1}, Q_{ϕ_2} and uses the minimum target:

$$y_t = r_t + \gamma \left(\min_{i \in \{1,2\}} Q_{\bar{\phi}_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi_{\theta}(a_{t+1} \mid s_{t+1}) \right)$$

Actor Objective

Actor Loss: $J_{\pi}(\theta) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_{\theta}} [\alpha \log \pi_{\theta}(a_t \mid s_t) - \min_{i=1,2} Q_{\phi_i}(s_t, a_t)]$

Target Update: Polyak averaging $\bar{\phi}_i \leftarrow \rho \phi_i + (1 - \rho) \bar{\phi}_i$. ρ is very small.

Soft Actor-Critic: Training Loop

1 Collect Experience:

$$a_t \sim \pi_\theta(\cdot \mid s_t), \quad (s_t, a_t, r_t, s_{t+1}) \rightarrow \mathcal{D}$$

2 Sample a Mini-batch:

$$(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}$$

3 Compute Target:

$$a_{t+1} \sim \pi_\theta(\cdot \mid s_{t+1})$$

$$y_t = r_t + \gamma \left[\min_{i=1,2} Q_{\phi_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\theta(a_{t+1} \mid s_{t+1}) \right]$$

4 Critic Update:

$$J_Q(\phi_i) = \mathbb{E} \left[\frac{1}{2} (Q_{\phi_i}(s_t, a_t) - y_t)^2 \right]$$

5 Actor Update:

$$a_t \sim \pi_\theta(\cdot \mid s_t)$$

$$J_\pi(\theta) = \mathbb{E} \left[\alpha \log \pi_\theta(a_t \mid s_t) - \min_{i=1,2} Q_{\phi_i}(s_t, a_t) \right]$$

Constrained Reinforcement Learning (CMDP)

Motivation for Constraints

Standard SAC maximizes reward without safety bounds. Real-world deployments (robotics, finance, energy grids) require explicit safety/budget guarantees:

- Energy expenditure, joint limits, torque limits, collision probability.

CMDP Optimization Problem

Let $C(s, a)$ be an environmental cost function with discounted return $J_C(\pi) = \mathbb{E}_{\tau \sim \pi} [\sum_{t=0}^{\infty} \gamma^t c_t]$.

$$\max_{\pi} J_{\text{SAC}}(\pi) \quad \text{subject to} \quad J_C(\pi) \leq d$$

where $d \geq 0$ is the prescribed maximum allowable cost threshold.

Lagrangian Dual Formulation

Unconstrained Min-Max Problem

We convert the constrained problem into a Lagrangian objective with multiplier $\lambda \geq 0$:

$$\max_{\pi} \min_{\lambda \geq 0} \mathcal{L}(\pi, \lambda) = J_{\text{SAC}}(\pi) - \lambda (J_C(\pi) - d)$$

Expanded Lagrangian Expectation

$$\mathcal{L}(\pi, \lambda) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (r_t - \alpha \log \pi(a_t | s_t) - \lambda c_t) \right] + \lambda d$$

- For a fixed multiplier λ , this matches SAC with modified reward: $\tilde{r}_t = r_t - \lambda c_t$.

CSAC Critic Architectures

CSAC decouples reward estimation from cost estimation by employing separate networks:

Reward Critics $Q_{\phi_i}^r$

Trained via clipped double-Q SAC loss:

$$y_t^r = r_t + \gamma \left(\min_{i=1,2} Q_{\phi_i}^r(s', a') - \alpha \log \pi_{\theta}(a' | s') \right)$$

$$L_r(\phi_i) = \mathbb{E} \left[(Q_{\phi_i}^r(s_t, a_t) - y_t^r)^2 \right]$$

Cost Critic Q_{ψ}^c

Estimates expected cumulative cost:

$$y_t^c = c_t + \gamma Q_{\psi}^c(s_{t+1}, a_{t+1})$$

$$L_c(\psi) = \mathbb{E} \left[(Q_{\psi}^c(s_t, a_t) - y_t^c)^2 \right]$$

CSAC Actor and Multiplier Optimization

Actor Objective in CSAC

The actor minimizes soft cost-penalized value:

$$J_{\pi}(\theta) = \mathbb{E}_{s_t, a_t \sim \pi_{\theta}} \left[\alpha \log \pi_{\theta}(a_t \mid s_t) - \min_{i=1,2} Q_{\phi_i}^r(s_t, a_t) + \lambda Q_{\psi}^c(s_t, a_t) \right]$$
$$\theta \leftarrow \theta - \alpha_{\pi} \nabla_{\theta} J_{\pi}(\theta)$$

Dual Ascent on Lagrange Multiplier λ

Projected subgradient ascent updates λ based on constraint violations:

$$\lambda \leftarrow [\lambda + \alpha_{\lambda} (J_C(\pi) - d)]_+, \quad \text{where } [x]_+ = \max(0, x)$$

- $J_C(\pi) > d \implies \lambda \uparrow$: Multiplier increases, forcing actor to prioritize safety.
- $J_C(\pi) < d \implies \lambda \downarrow$: Multiplier decreases, allowing actor to prioritize reward.

CSAC Training Loop: Environment and Critics

1 Collect transition

$$a_t \sim \pi_\theta(\cdot \mid s_t) \\ (s_t, a_t, r_t, c_t, s_{t+1})$$

2 Update twin reward critics

$$y_t^r = r_t + \gamma \left[\min_{i \in \{1,2\}} Q_{\phi_i}^r(s_{t+1}, a_{t+1}) - \alpha \log \pi_\theta(a_{t+1} \mid s_{t+1}) \right]$$

$$L_r(\phi_i) = \mathbb{E} \left[(Q_{\phi_i}^r(s_t, a_t) - y_t^r)^2 \right]$$

$$\phi_i \leftarrow \phi_i - \alpha_\phi \nabla_{\phi_i} L_r(\phi_i) \quad \text{for } i \in \{1, 2\}$$

3 Update cost critic

$$y_t^c = c_t + \gamma Q_\psi^c(s_{t+1}, a_{t+1})$$

$$L_c(\psi) = \mathbb{E} \left[(Q_\psi^c(s_t, a_t) - y_t^c)^2 \right]$$

$$\psi \leftarrow \psi - \alpha_\psi \nabla_\psi L_c(\psi)$$

CSAC Training Loop: Actor Update

Actor uses both reward and cost critics

$$J_{\pi}(\theta) = \mathbb{E}_{s_t, a_t \sim \pi_{\theta}} \left[\alpha \log \pi_{\theta}(a_t \mid s_t) - \min_i Q_{\phi_i}^r(s_t, a_t) + \lambda Q_{\psi}^c(s_t, a_t) \right]$$

- $-Q^r$: maximize expected reward.
- $\alpha \log \pi$: retain SAC entropy / exploration.
- $+\lambda Q^c$: penalize high-cost actions.

$$\theta \leftarrow \theta - \alpha_{\pi} \nabla_{\theta} J_{\pi}$$

CSAC Training Loop: Dual and Target Updates

1 Update Lagrange multiplier

$$\lambda \leftarrow [\lambda + \alpha_\lambda (J_C(\pi) - d)]_+$$

- $J_C > d$: constraint violated $\Rightarrow \lambda \uparrow$
- $J_C < d$: constraint satisfied $\Rightarrow \lambda \downarrow$

2 Update target critics

$$\bar{\phi}_i \leftarrow \rho \phi_i + (1 - \rho) \bar{\phi}_i$$

$$\bar{\psi} \leftarrow \rho \psi + (1 - \rho) \bar{\psi}$$

3 Repeat

Collect $\rightarrow$ Critics $\rightarrow$ Actor $\rightarrow \lambda \rightarrow$ Targets $\rightarrow$ Repeat